

Cloud Computing Project

FS 2025/2026, Shao Jü Woo

Assignment description:

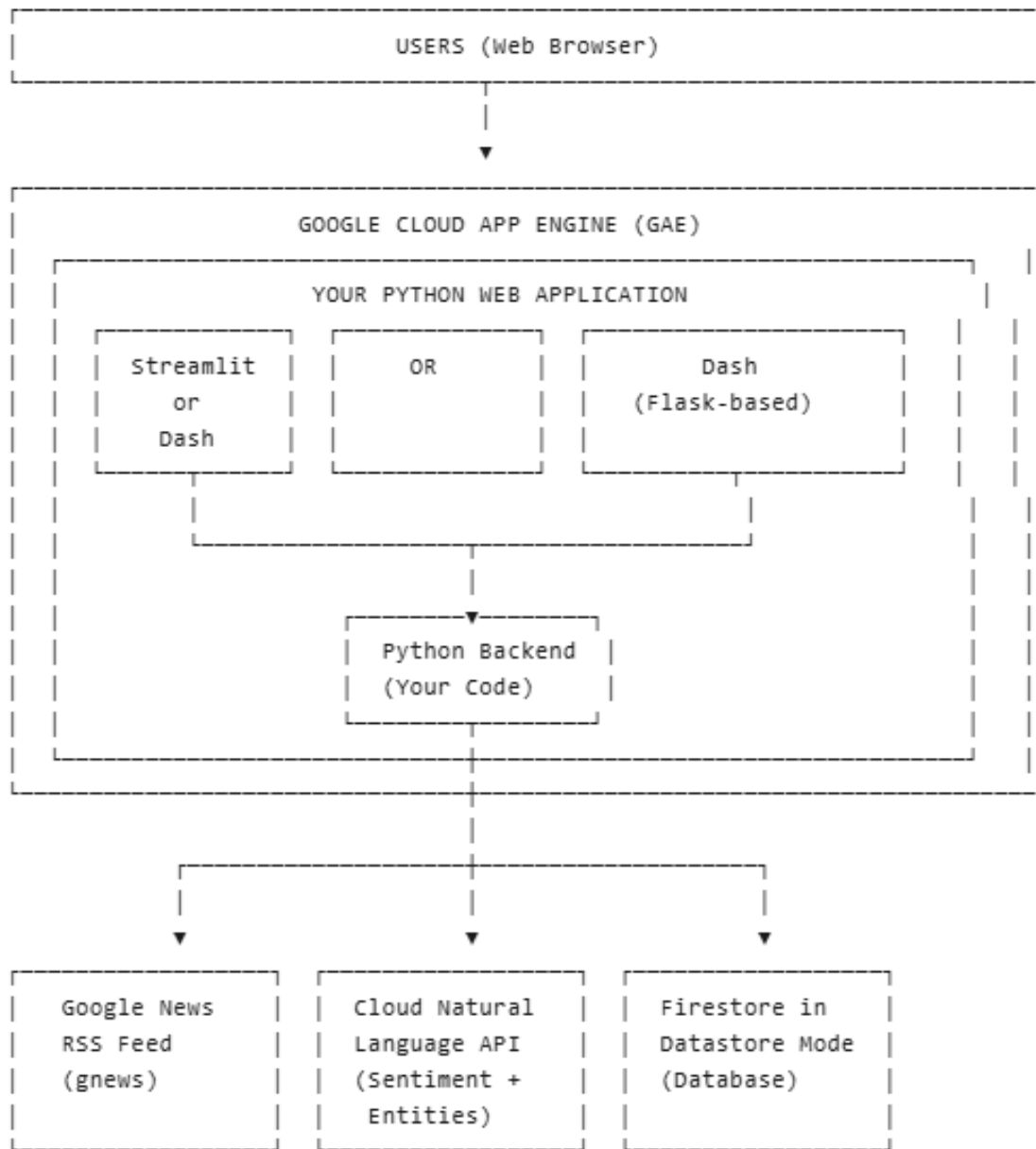
In this project, a reactive web application for analyzing text contained in articles related to **companies** has to be designed and implemented in **Python on Google Cloud Platform**.

Text analysis of company related news can aid stock investors and traders by monitoring market sentiment, identifying trends, and assessing risks. In general, by analyzing news, it provides insights into company stocks, enhancing decision-making and risk management in the market.

The application runs on **Google's App Engine (GAE)** and will be designed using either Streamlit or Dash which is an extension of Flask. This project will enable you to gain practical experience in cloud computing, modern AI techniques, and web development by implementing sentiment analysis of articles related to companies. The project utilizes Google Cloud Platform services and libraries, specifically:

- **Google Cloud Natural Language API** (updated from Vertex AI Studio) for text analysis using Natural Language Processing (NLP) - including sentiment analysis, entity extraction, and content classification
- **Gemini in Vertex AI** (**optional**/advanced) for generative AI-based text summarization and advanced entity extraction using prompt engineering
- **GCP App Engine** (Python 3.12 runtime) for running web-based applications
- **Streamlit** (v1.42+) or **Dash** (v2.18+) for creating reactive, GUI-based web applications
- **Firestore in Datastore mode** (updated from legacy Datastore) for storing data - note that Datastore is now running on Firestore infrastructure with full backward compatibility
- **Google News RSS Feed** via gnews Python package to fetch news from various sources
- **newspaper3k** library (v0.2.8) for article text extraction

System Architecture:



1. Development Workflow: From Local Development to Cloud Deployment

This section describes the recommended workflow for developing your application locally on your PC and then deploying it to Google Cloud App Engine.

Phase 1: Local Development

Develop and test your application on your local machine using real Google Cloud services.

Step 1: Set Up Local Environment *inip- fs 26*

- ✓ Install Required Software:
- ✓ Python 3.11 or 3.12 (from python.org)
- ✓ Visual Studio Code or PyCharm (IDE)
- ✓ **Installation of Required Libraries:**

```
✓ pip install google-cloud-language>=2.16.0
pip install google-cloud-datastore>=2.21.0
pip install gnews
pip install newspaper3k>=0.2.8
pip install streamlit>=1.42.0 # or pip install dash>=2.18.0
pip install pandas plotly
```

Step 2: Obtain Google Cloud Credentials

- ✓ Go to Google Cloud Console
- ✓ Create a new project (e.g., financial-news-analyzer)
- Enable APIs:
 - Cloud Natural Language API
 - Cloud Datastore API (Firestore in Datastore mode)
 - Vertex AI API (**optional**, if using Gemini)
- Setting Google Cloud Application Credentials
 - When developing your application in a **local** development environment, you'll interact with Google Cloud Platform services like Google Cloud **Firestore / Datastore** and **Cloud Natural Language API** via your Python program. To start, you must authenticate the application correctly and grant all necessary permissions:

Konto...

Step 1: Get a service account key from Google Cloud Console

1. Go to [Google Cloud Console](#)
2. Create a project (or select existing)

3. Go to "IAM & Admin" → "Service Accounts"
4. Create a service account
5. Grant roles: Cloud Datastore User, Cloud Datastore Owner, Cloud Natural Language API User
6. Create a key (JSON format) and download it:

You'll get a json key file like: `my-project-123456-abcdef.json`

What you must also do for Firestore are these additional steps:

7. Go to **APIs & Services** → **Library**
8. Enable **Cloud Firestore API**

And the first time you open Firestore, Google will ask you to **create the database**.

Step 2: Tell your computer where this file is by setting `GOOGLE_APPLICATION_CREDENTIALS` environment variable to the path of your JSON key file so that your Python program can locate the credentials (see example Python code in 3.1 Sentiment Analysis).

Step 3: Configure Local Authentication

Create a file `config.py` in your project root

Step 4: Build Core Application Locally

Create `main.py` (Streamlit)

Step 5: Run Locally

Access at `http://localhost:8501`

Verify in Cloud Console:

- Go to [Datastore Viewer](#)
- Confirm data is being stored in Datastore

Phase 2: Deploy to App Engine

Refer to «GCP Übung: Erste Anwendung mit Appengine erstellen» (on Moodle)

2. Google News RSS Feed

The Google News RSS Feed is a service provided by Google News that allows users to retrieve news from various sources in a standardized format called RSS (Really Simple Syndication).

A lightweight Python Package that searches Google News RSS Feed and returns a usable JSON response is available: [gnews · PyPI](#). This package allows setting specifications for the search results by passing proxy, country, language, period, start date, end date exclude websites and size during initialization.

As an example, the following code searches for news on the company NVIDIA (ticker symbol: NVDA) by using a keyword "nvda":

```
from gnews import GNews

# Initialize with search parameters
google_news = GNews(
    language='en',
    country='US',
    period='1d',
    max_results=10
)

# Search for news by keyword
retrieved_news = google_news.get_news('nvda')

# Check if results exist before accessing
if retrieved_news:
    print(f"Found {len(retrieved_news)} articles")
    print(retrieved_news[0])
else:
    print("No news found")
```

The corresponding output:

```
Found 10 articles
{'title': 'NVDA to Take Over Do North Coworking in Lyndonville - The North Star Monthly',
 'description': 'NVDA to Take Over Do North Coworking in Lyndonville The North Star Monthly',
 'published date': 'Thu, 05 Mar 2026 20:48:00 GMT',
 'url': 'https://news.google.com/rss/articles/CBMi1gFBVV95cUxOdExHdmlXTGxJdEFFckQ4U3BLdXpsbmdwaUZrWmpVMHh30EV0eXpuOXUyMEZzX0p6VVdyLTlQaExqekU2YnoxRmV1dkhiMnZDaEJhWjBswWlpbEd2N3NrNFYxLVV5OHJKdDBTbGo3VmFec0dBUjZMajExcTNwUTVpX2hnS1RMcEVdX0RVX0tpX0FHUlpGSWZrSm1uNDRFamluT0RVd1YwWjYzVVdnM0lpZWNhTE5DZk5HcUM5UVFMS1FqOXo2cUJUdE1qai1VYUFJVkhLVjRB?oc=5&hl=en-SG&gl=SG&ceid=SG:en
```

Article Extraction: The link [gnews · PyPI](#) also describes how the **full article** can be fetched by either navigating to the **url** directly in your browser or by using the newspaper3k library (v0.2.8) to scrape full article content. This eliminates the need for users to develop custom article scrapers.

```
from newspaper import Article

# URL from GNews results
url = "https://www.investopedia.com/..." # Use actual URL from retrieved_news

article = Article(url)
article.download()
article.parse()

# Check if download succeeded before accessing content
if article.is_downloaded:
    print("Title:", article.title)
    print("Text:", article.text[:500], "...") # First 500 characters
    print("Authors:", article.authors)
    print("Publish Date:", article.publish_date)
else:
    print("Failed to download article")
```

3. Text Analysis using Google Cloud Natural Language API

[Google Cloud Natural Language API](#) provides powerful, pre-trained machine learning models for understanding text. For this project, you will need to set up

3.1 Sentiment Analysis

Analyzes text and determines the overall sentiment (positive, negative, or neutral) with confidence scores ranging from -1.0 (negative) to +1.0 (positive), plus magnitude scores indicating emotional strength.

Working Example:

```
# import Python Library module for using the Natural Language API.
from google.cloud import language_v1
import os

def analyze_sentiment(text: str):
    """
    Analyzes sentiment of the provided text.
    Returns score (-1.0 to +1.0) and magnitude (0.0 to infinity).
    """

    # Manually set the path BEFORE creating the client
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = r"C:\path\to\your\key-file.json"

    # Initialize client (uses GOOGLE_APPLICATION_CREDENTIALS environment variable)
    client = language_v1.LanguageServiceClient()

    # Create document
    document = language_v1.Document(
        content=text,
        type_=language_v1.Document.Type.PLAIN_TEXT
    )

    # Analyze sentiment (v2 API style with request dict)
    response = client.analyze_sentiment(request={"document": document})

    # Get overall sentiment
    sentiment = response.document_sentiment

    results = {
        "score": sentiment.score,
        "magnitude": sentiment.magnitude,
        "sentences": []
    }

    # Analyze sentence-level sentiment
```

```
for sentence in response.sentences:
    results["sentences"].append({
        "text": sentence.text.content,
        "score": sentence.sentiment.score,
        "magnitude": sentence.sentiment.magnitude
    })

return results

# Example usage
text = """
Apple's earnings exceeded expectations this quarter.
However, supply chain concerns remain in Asian markets.
"""

result = analyze_sentiment(text)
print(f"Overall Score: {result['score']}")           # 0.6 (positive)
print(f"Overall Magnitude: {result['magnitude']}")   # 3.2 (strong emotion)

for sent in result['sentences']:
    print(f"Sentence: {sent['text']}")
    print(f"  Score: {sent['score']}, Magnitude: {sent['magnitude']}")
```

This outputs:

```
Overall Score: 0.0
Overall Magnitude: 1.399999976158142
Sentence: Apple's earnings exceeded expectations this quarter.
  Score: 0.699999988079071, Magnitude: 0.699999988079071
Sentence: However, supply chain concerns remain in Asian markets.
  Score: -0.6000000238418579, Magnitude: 0.6000000238418579
```

Interpretation Guide:

- Score > 0: Positive sentiment
- Score < 0: Negative sentiment
- Score $\approx$ 0: Neutral
- Magnitude > 2.0: Strong emotions expressed (regardless of positive/negative)

3.2 Entity Extraction (Named Entity Recognition)

Entity extraction, or named entity recognition (NER), is a vital task in natural language processing (NLP). It involves identifying and categorizing named entities that appear in text, such as names of technologies, medications, people, organizations, locations, dates, and more. The aim is to automatically find and classify entities in text to provide structured information. This task is essential for various NLP applications such as information retrieval and question answering.

The entity types for financial news are:

- **ORGANIZATION:** Companies, institutions, and other organizations.
- **PERSON:** Individuals mentioned in the article (e.g. Elon Musk).
- **TICKER SYMBOL:** The stock market ticker symbol for a company (while not always explicitly present in the text, it's a very important entity to infer when analyzing stock-related articles).
- **DATE:** Dates and time periods.
- **LOCATION:** Places mentioned in the article (e.g., headquarters locations, regions affected by news).
- **EVENT:** Significant events mentioned in the article (e.g., earnings reports, mergers, product launches, economic events).
- **INDUSTRY:** The sector or industry the company belongs to (e.g., technology, finance, healthcare).

An example of entities extracted from a news article related to Bank of America:

- **ORGANIZATION:** Federal Reserve
- **EVENT:** Rate Hike
- **DATE:** Next Week
- **INDUSTRY:** Financial Stocks
- **ORGANIZATION:** Bank of America
- **TICKER SYMBOL:** BAC (Implied, i.e. inferred)

Tips for extracting entities:

- Call the correct API method: Once you have created the client and the document object, you need to send the document to the Natural Language API. Look for a method of the client that analyzes entities in a document.
- The API returns a response object that contains the results of the analysis. The response object contains a list of detected entities. Iterate through the entities: using a for loop to go through each detected entity

4. Library for creating Reactive Web Apps in Python

The front-end of the web application can be designed using either Streamlit or Dash.

4.1 Streamlit (Recommended for Beginners)

Streamlit is a user-friendly library designed for Python developers to create interactive web applications with ease. It simplifies the process of building web interfaces, allowing you to focus more on your project's logic and less on the complexities of web development.

With Streamlit, you can turn relatively quickly your Python scripts into functional web apps.

An example of a concrete Streamlit application can be found here: [How to Build a Custom Summarization App with LangChain and Streamlit | by Lucas Soares | MLearning.ai | Jun, 2023 | Medium](#)

4.2 Dash:

Dash is an open-source user interface library for creating reactive, Web-based applications in **Python**. Written on top of [Flask](#), Plotly.js and React.js, Dash is well-suited for **quickly** building customized web applications. Dash **extends** Flask: every time we create a Dash app, we are essentially developing a Flask app with additional features and enhancements.

Dash can be downloaded from Python's package manager using the command `pip install dash`. A *getting started* guide can be found [here](#) and the Dash code on GitHub [here](#).

Dash makes it simple to build a GUI around your data analysis code which can be viewed in the web browser. A special feature of Dash is that you don't have to write any Javascript or HTML. Dash provides a Python interface to a rich set of interactive web-based components. For a gallery of Dash apps refer to: [Dash Enterprise](#)

An example of a concrete Dash application can be found [here](#).

In comparison, Dash can be seen as an extended version of Streamlit that offers similar functionalities. This means that everything you can implement with Streamlit can theoretically also be realised with Dash. However, the implementation in Dash may require a little more time and effort compared to Streamlit.

5. Google Cloud App Engine for Deploying the Dash application

App Engine is a fully managed serverless platform service that can be used to deploy highly scalable web applications: [Python on Google App Engine | App Engine Documentation | Google Cloud](#)

Current Runtime Support:

- Python 3.12 (recommended, supported until October 2028)
- Python 3.13 (supported until October 2029)
- Python 3.11 (supported until October 2027)

Important: Python 2.7 reached end-of-life in January 2026 and should NOT be used.

6. Firestore in Datastore Mode for Storing and Reading Data

The web application connects to and handles data through Firestore in [Datastore mode](#). (formerly Google Cloud Datastore). This is a non-relational (NoSQL) database built for automatic scaling, high performance, and ease of application development. Use the [Datastore client libraries](#) to connect your web service to Datastore.

7. Project requirements:

The web application should meet the following requirements:

1. **News Search:** The user should be able to search for news related with a company through keywords. The GUI could be extended by setting other search parameters such the topic: the available topics are WORLD, NATION, BUSINESS, TECHNOLOGY, ENTERTAINMENT, SPORTS, SCIENCE, HEALTH. The function call for topic-based search is given by: `GNews.get_news_by_topic(topic)`. Note: in this case the function does not include the company related search keyword.
2. **Data Storage:**
 - For each retrieved article, the application stores the analyzed data in Datastore. This includes the full article text, computed sentiment scores, and the extracted named entities. The stored data should be available for future reference and analysis.
 - The application must support querying Datastore using appropriate filters. For example, users should be able to retrieve articles that mention a specified company or companies within a particular industry sector, have sentiment scores above a specified threshold, and were published within a defined date range.
 - **Error Handling:** The system should implement appropriate error handling mechanisms, including a fallback strategy for cases where article URLs are unreachable or the article content cannot be retrieved.

3. **Visualization:** Using Streamlit or Dash create suitable **visualizations** of the various analysis results or data retrieved from Datastore, e.g., in the form of charts. For inspiration refer to [Dash Enterprise App Gallery](#).
4. **Data Export:** Implement a function in the Web application to export data (query results) using a suitable file format (CSV, JSON, or Excel).

Submission deadline of project: 19. June 2026

Submission: Code + short documentation.

Exam: online presentation via MS Teams in June/July 2026

Grading:

The grading criteria for the Cloud Computing project can be broken down as follows:

- Functionality, i.e. does the app run, 50%
- Functional scope, 20%
- Quality of design and implementation, 20%
- Creativity, 10%

Note: If your GCP credits have run out, please contact me via email, and I will arrange for additional GCP credits to be provided.

Official Documentation:

- [Cloud Natural Language API](#)
- [Vertex AI Gemini](#)
- [Firestore Datastore Mode](#)
- [App Engine Python 3](#)